

2.1 TypeScript

TypeScript es lo que se conoce como un superset de Javascript, es Javascript con nuevas utilidades que lo convierten en un lenguaje más completo. Entre otras cosas, lo más particular de TypeScript es que incluye tipos en el lenguaje Javascript, para convertirlo en un lenguaje fuertemente tipado además de mecanismos propios de POO(interfaces)

Un superset es un lenguaje “escrito encima de otro” o mejor dicho, que compila a otro lenguaje. En el caso de TypeScript es un lenguaje que compila a JavaScript, pero que incluye muchas facilidades y ventajas.

Los lenguajes como JavaScript basados en un estándar evolucionan muchas veces más lento que las necesidades de los desarrolladores. Entonces surgen empresas o comunidades que deciden expandir un lenguaje, aportando todas las herramientas que se echan en falta para poder desarrollar con las mejores condiciones.

Los superset compilan en el lenguaje estándar, por lo que el desarrollador programa en aquel lenguaje expandido, pero luego su código es "transpilado" para transformarlo en el lenguaje estándar, capaz de ser entendido en todas las plataformas.

La característica fundamental de TypeScript es que compila en Javascript nativo, por lo que se puede usar en todo proyecto donde se esté usando Javascript. Dicho con otras palabras, cuando se usa TypeScript en algún momento se realiza su compilación, convirtiendo su código a Javascript común. El navegador, o cualquier otra plataforma donde se ejecuta Javascript, nunca llegará a enterarse que el código original estaba escrito en TypeScript, porque lo único que llegará a ejecutar es el Javascript resultante de la compilación.

Uno de los entornos donde utilizaremos TypeScript es Angular, ya que Angular 2 (y las versiones sucesivas) lo usan como lenguaje.

Definición de tipos

A los tipos primitivos de javascript boolean, bigint, null, number, string, symbol, object y undefined les añade any(permitir cualquier cosa), unknown(asegúrese de que alguien que usa este tipo declare cuál es el tipo), never(no es posible que este tipo pueda suceder) y void (una función que devuelve undefined o no tiene retorno valor).

Declaración de variables

let Nombre: tipo = valor;

Es válido nombre=valor

Sin indicar tipos puesto que se compila código javascript, aunque no es lo propio de typescript puesto que su filosofía es más bien la contraria. Para estos casos tenemos el tipo any, lo utilizaremos cuando no queramos dar tipo a una variable. Obviamente no es necesario inicializar la variable.

Definición de arrays

Para definir un Array en TypeScript podemos hacerlo de dos formas. La primera es indicando el tipo de los elementos del array y añadiendo el operador [].

```
miarray: tipodato[];
```

Por ejemplo podríamos definir un array de números de la siguiente forma:

```
miarray: number[] = [1,2,3,4,5];
```

Dentro del operador <> se indica el tipo de dato que vamos a utilizar. En este caso, para definir un array de números sería:

Para acceder a los elementos array[posición]

Para añadir un elemento a un array método push (investigar otros métodos)

Declaración de objetos: clases - interfaces

Clases

Como en cualquier otro lenguaje de programación orientado a objetos, en TypeScript las clases tienen campos, constructores, propiedades y funciones. La sintaxis es similar a la de todos los lenguajes orientados a objetos.

```
class Car {  
    engine: string;  
  
    constructor(engine: string) {  
        this.engine = engine;  
    }  
}
```

Field

Constructor

Shorthand way to declare a field

```
class Car {  
    constructor(public engine: string) { }  
}
```

Los parámetros de entrada pueden ser tipados o no, tampoco es obligatorio indicar el tipo de valor que retornan los métodos

```
class Car {  
    engine: string;  
  
    constructor (engine: string) {  
        this.engine = engine;  
    }  
  
    start() {  
        return "Started " + this.engine;  
    }  
  
    stop() {  
        return "Stopped " + this.engine;  
    }  
}
```

Class members are public by default

También podemos tener gets y sets

```
class Car {  
    private _engine: string;  
  
    constructor(engine: string) {  
        this.engine = engine;  
    }  
  
    get engine(): string {  
        return this._engine;  
    }  
  
    set engine(value: string) {  
        if (value == undefined) throw 'Supply an Engine!';  
        this._engine = value;  
    }  
}
```



Properties act as filters and can have get or set blocks

Interfaces

Una interfaz se define con la palabra interface y el nombre. Las interfaces no son solamente para funciones, también pueden ser para propiedades sólo.

Podemos indicar elementos opcionales, es decir no es obligatorio implementarlo. En este caso, les añadimos el signo de interrogación

Optional Members in an Interface

```
interface IAutoOptions {  
    engine: IEngine;  
    basePrice: number;  
    state: string;  
    make?: string;  
    model?: string;  
    year?: number;  
}
```



Optional Members

Para inicializar una variable de un tipo de interfaz

```
let auto:IAutoOptions={} as IAutoOptions
```

Decoradores

Un decorador es una función que, dependiendo de qué cosa queramos *decorar*, sus argumentos serán diferentes. Usan la forma `@expression` donde `expression` evaluará la función que será llamada. Los decoradores más frecuentes:

Decorador de clase

Se aplica al constructor de la clase y puede ser usado para observar, modificar o reemplazar la definición inicial de la clase.

Decorador de métodos

Tienen el mismo objetivo que las clases de observar, modificar o reemplazar.

Decorador de propiedades y parámetros

Son más simples que los anteriores, un decorador de propiedades debe tomar como parámetros `target`, que es el prototipo de la clase, y `propertyKey`, el nombre de la propiedad. Finalmente, los decoradores de parámetros funcionan igual, pero con el parámetro adicional `parameterIndex`, que indica la posición en el array, por ejemplo:

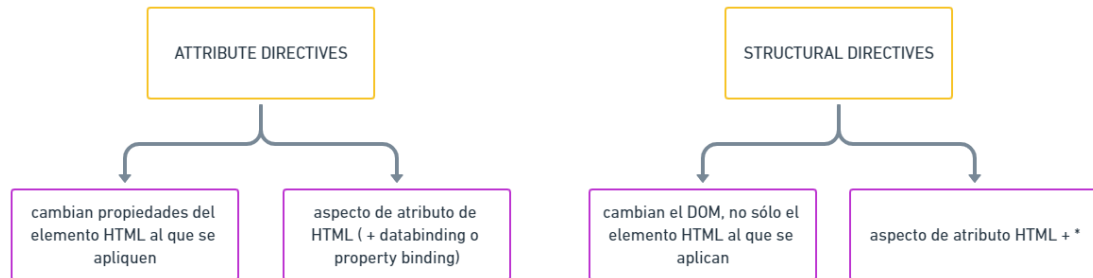
Directivas

Las directivas (en inglés, *directives*) son una pieza clave para poder construir aplicaciones con Angular. Son, esencialmente, instrucciones para manipular el DOM.

De hecho, los componentes son un tipo de directiva, ya que cuando usamos el selector de un componente, le estamos pidiendo a Angular que muestre dicho componente (y su lógica de programación) en algún lugar determinado del DOM.

Son invocadas cuando el DOM (Document Object Model) es compilado por el framework de Angular. Su finalidad es modificar o crear un comportamiento totalmente nuevo.

Hay dos tipos de directivas en angular



1. Directivas Estructurales

***ngIf**

Una directiva estructural que evalúa una expresión. Cuando la expresión se evalúa como verdadera, Angular representa la plantilla proporcionada en una cláusula then, y cuando es falsa o nula, Angular representa la plantilla proporcionada en una cláusula else opcional.

Ejemplos:

```
<p *ngIf="nota>=5">has aprobado</p>
```

```
<p *ngIf="nota>=7 && nota<9">notable</p>
```

La condición puede implementarse en una función dentro del componente dueño de la plantilla dónde escribimos las directivas, siempre que la lógica se complique será mejor esta opción

```
<p *ngIf="esMuyListo()">estudia aeronautica</p>
```

En clase del componente definimos la función:

```
export class NotaComponent implements OnInit {  
  
  nota:number;  
  asignatura:string;  
  edad:number;  
  constructor() {  
    this.asignatura="matematicas";  
    this.nota=5;  
    this.edad=16;  
  }  
  
  esMuyListo():boolean {  
    if((this.nota==9 & this.asignatura=='matematicas') || this.edad==16 ) {  
      return true;  
    }else {  
      return false;  
    }  
  }  
  
  ngOnInit() {  
  }  
}
```

Veremos ejemplos prácticos en angular

***ngFor**

La directiva ngFor es la que se encarga de presentar una lista de elementos en pantalla de una forma sencilla combinando el concepto de bucle y plantilla. Esta directiva soporta muchas opciones, vamos a verlas

En su versión más sencilla recorre una lista de elementos sin índice

lista:string[]=["hola", "que", "tal", "estas"];

```
<table>
```

```
<tr *ngFor="let item of lista">
```

```
<td>
```

```
{{item}}
```

```
</td>
```

```
</tr>
```

```
</table>
```

Podemos incluir el índice de los elementos

```
<table>
```

```
<tr *ngFor="let item of lista;let indice=index">
```

```
<td> {{indice+1}}
```

```
<td>{{item}}</td>
```

```
</tr>
```

```
</table>
```

Pipes

Los pipes son filtros o funciones que pones directamente en la vista para el dar formato a un dato que estés pintando

Cada vez que desarrollamos una aplicación mostramos información a partir de variables que recogen datos, como el nombre de un usuario, una fecha, un porcentaje... Pero no siempre la información que mostramos viene directamente con el aspecto final con el que el usuario se encontrará, como por ejemplo la fecha de hoy:

hay muchos más pipes que podemos utilizar en ionic gracias a angular, para modificar fechas, cadena de caracteres, números y más.